

Task 4 — Horizontal Scale & Load Readiness

Data Analyst · Sprint A - Scale & Reliability · Task Brief

DATA ANALYST

PlaceMux · Phase 3 · Task Brief

TASK 4 / 25

Advanced · Advanced

Sprint A - Scale & Reliability · Mastery 65%

Focus

Model scale: forecast traffic, load and cost so capacity planning is evidence-based.

What to build / complete

- A demand forecast (traffic, applications, peaks like placement season)
- A capacity and cost projection at 2x/5x/10x scale
- Documented assumptions and the risk of being wrong

Execution — work through in order

Phase 3 gives you a goal and a bar, not a recipe. These stages are the discipline; the design decisions are yours to make and defend.

Stage A — Understand & set the bar

1. Re-read the Expected Output and the scoring parameters so you build to the bar, not past it.
2. Confirm every prerequisite and upstream input is actually in your hands, not merely promised.
3. Restate, in one sentence, what 'good' means here: The team scales for the traffic that is actually coming, not the traffic they imagine.

Stage B — Build: A demand forecast (traffic, applications, peaks like placement season)

1. Define the metric, its source & the decision

Define precisely what “A demand forecast (traffic, applications, peaks like placement season)” measures, which raw events it depends on, and which decision it changes. No decision, no metric.

2. Build the query/model on real data

Build the query/model on real production data — sampled if you must, but never synthetic.

3. Validate: reconcile, sanity-check, quantify uncertainty

Reconcile against a source system, sanity-check the magnitude, and state the uncertainty honestly.

4. Tie every number to an action & defend it

For every number, be ready to say where it came from and what action it triggers. Undefendable numbers do not ship.

Stage C — Build: A capacity and cost projection at 2x/5x/10x scale

1. Define the metric, its source & the decision

Define precisely what “A capacity and cost projection at 2x/5x/10x scale” measures, which raw events it depends on, and which decision it changes. No decision, no metric.

2. Build the query/model on real data

Build the query/model on real production data — sampled if you must, but never synthetic.

3. Validate: reconcile, sanity-check, quantify uncertainty

Reconcile against a source system, sanity-check the magnitude, and state the uncertainty honestly.

4. Tie every number to an action & defend it

For every number, be ready to say where it came from and what action it triggers. Undefendable numbers do not ship.

Stage D — Build: Documented assumptions and the risk of being wrong

1. Define the metric, its source & the decision

Define precisely what “Documented assumptions and the risk of being wrong” measures, which raw events it depends on, and which decision it changes. No decision, no metric.

2. Build the query/model on real data

Build the query/model on real production data — sampled if you must, but never synthetic.

3. Validate: reconcile, sanity-check, quantify uncertainty

Reconcile against a source system, sanity-check the magnitude, and state the uncertainty honestly.

4. Tie every number to an action & defend it

For every number, be ready to say where it came from and what action it triggers. Undefendable numbers do not ship.

Stage E — Integrate, verify & make demoable

1. Wire your work into the end-to-end flow and run the whole journey once, start to finish.
2. Present the forecast with backtested accuracy and clearly stated assumptions.
3. Break it on purpose: force the failure path and confirm it degrades the way you designed.
4. Prepare a live demo on real data. Not slides, not screenshots, not 'it works on my machine'.

Done when (demoable by end of task)

- “A demand forecast (traffic, applications, peaks like placement season)” is complete, real, and demoable end-to-end.
- “A capacity and cost projection at 2x/5x/10x scale” is complete, real, and demoable end-to-end.
- “Documented assumptions and the risk of being wrong” is complete, real, and demoable end-to-end.
- Good looks like: The team scales for the traffic that is actually coming, not the traffic they imagine.

Worry if any of these are true

- A forecast with no backtest.
- Ignoring seasonality in a seasonal business.
- Calling an experiment early because it looked good on day two.

Hand-off

- Forecast to DevOps capacity planning. Make sure the next person can pick it up without you.